

MCDI500 PROGRAMACIÓN PARA LA CIENCIA DE DATOS PROYECTO FINAL

INTEGRADOR – FASE 4

**PATRONES TEMPORALES DE GENERACIÓN ELÉCTRICA EN CENTRALES TER CMPC LAJA,
PACÍFICO Y SANTA FE – ENERO A AGOSTO 2026**

Nombre integrante/s:

Durán Cisterna, Verónica

Moya Arriagada, Raúl

Rojas Vilches, Daniela

Sánchez Cárcamo, Manuel

Curso:

MCDI500 - Programación para la Ciencia de Datos

Docente:

Dr. Omar Salinas Silva

URL repositorio:

<https://github.com/vedurancisterna-alt/Proyecto-generacion-electrica-centrales-CMPC>

Fecha:

4 de octubre de 2026

Contenido

I. Portada e índice	1
II. Introducción y contextualización.....	3
III. Definición de la problemática y objetivos del proyecto.....	3
3.1 Problemática y pregunta de investigación.....	3
3.2 Objetivo general.....	3
3.3 Objetivos específicos	3
3.4 Alcance, restricciones y supuestos	4
3.5 Articulación con las fases F1–F4	4
IV. Aplicación de herramientas científicas	4
a. Reproducibilidad técnica.....	4
b. Control de versiones.....	4
c. Documentación del proceso.....	5
d. Repositorio GitHub actualizado.....	5
V. Diseño de soluciones algorítmicas eficientes	5
a. Codificación funcional	5
b. Preprocesamiento y transformación del dataset	5
c. Validación técnica y verificación del código.....	5
d. Eficiencia y optimización.....	5
e. Diseño estructurado del código.....	6
f. Notebook final ejecutado y documentado	6
VI. Implementación de código modular y robusto	6
a. Programación orientada a objetos	6
b. Documentación de arquitectura	6
VII. Construcción de visualizaciones de datos efectivas.....	7
a. Preprocesamiento y transformación de datos	7
b. Visualizaciones analíticas	7
VIII. Metodología del desarrollo técnico.....	8
a. Validación y reflexión técnica.....	8
b. Resultados.....	9
c. Discusión	9
d. Conclusiones	9
e. Trazabilidad de mejoras.....	9
IX. Presentación Audiovisual.....	10
X. Bibliografía.....	10

II. Introducción y contextualización

El proyecto analiza datos públicos de Generación Real publicados por el Coordinador Eléctrico Nacional (CEN) para las centrales TER CMPC Laja, TER CMPC Pacífico y TER CMPC Santa Fe durante el período enero-agosto de 2026. El propósito es caracterizar cómo varía la generación eléctrica en el tiempo, identificar patrones horarios y mensuales y comparar el comportamiento observado entre las tres centrales. La información utilizada proviene de una fuente oficial y se interpreta de acuerdo con el diccionario de datos del Sistema de Información Pública del CEN (Coordinador Eléctrico Nacional, 2022; 2026).

La necesidad técnica surge del volumen de los registros: la fuente original tiene 359.891 filas y 33 columnas, y llevarla a 17.496 observaciones analíticas (una por central, fecha y hora) exige programación para automatizar la preparación, la validación y el análisis de forma reproducible.

La relevancia del análisis es práctica: quienes operan estas plantas de cogeneración o analizan su comportamiento de generación eléctrica pueden usar los patrones horarios, mensuales y la frecuencia de 0 MWh para dimensionar la variabilidad de la generación y priorizar qué períodos revisar con información operacional. El trabajo se enmarca en el curso de Programación para la Ciencia de Datos (Salinas Silva, 2026b) y se organiza en cuatro fases: F1 definió el problema y el entorno, F2 preparó y validó los datos, F3 construyó el núcleo algorítmico (POO, eficiencia y recursividad) y F4 integra, visualiza y comunica los resultados (Tabla de la sección 3.5).

III. Definición de la problemática y objetivos del proyecto

3.1 Problemática y pregunta de investigación

La problemática consiste en describir y comparar el comportamiento temporal de la generación eléctrica registrada por las centrales seleccionadas, utilizando exclusivamente la evidencia disponible en los datos públicos del CEN. La pregunta que guía el análisis se mantiene sin cambios desde la Fase 1: ¿Qué patrones temporales de generación eléctrica caracterizan a las centrales TER CMPC Laja, TER CMPC Pacífico y TER CMPC Santa Fe durante el período enero-agosto de 2026?

La selección de TER CMPC Laja, TER CMPC Pacífico y TER CMPC Santa Fe responde a que conforman el conjunto de centrales termoeléctricas de cogeneración a biomasa de CMPC Pulp S.A. Estas plantas generan energía principalmente para autoabastecer sus propios procesos de celulosa según su programación interna, y comercializan los excedentes inyectándolos al Sistema Eléctrico Nacional.

3.2 Objetivo general

Caracterizar los patrones temporales de generación eléctrica de las centrales TER CMPC Laja, TER CMPC Pacífico y TER CMPC Santa Fe durante enero-agosto de 2026, utilizando datos públicos del Coordinador Eléctrico Nacional mediante un proceso reproducible de preparación, validación y análisis.

3.3 Objetivos específicos

- Caracterizar la distribución horaria, diaria y mensual de la generación eléctrica de las tres centrales.
- Comparar los patrones temporales observados entre TER CMPC Laja, TER CMPC Pacífico y TER CMPC Santa Fe.
- Analizar la frecuencia y la distribución temporal de los registros con generación igual a 0 MWh, sin

atribuirles causas operacionales que no estén respaldadas por la fuente.

- Identificar regularidades y diferencias que sustenten la comunicación de resultados y las conclusiones finales del proyecto.

3.4 Alcance, restricciones y supuestos

- **Alcance.** Tres centrales (Laja, Pacífico y Santa Fe), del 1 de enero al 31 de agosto de 2026, con análisis descriptivo de patrones horarios, diarios y mensuales y de los registros de 0 MWh. No incluye otras centrales, pronósticos ni causas operacionales.
- **Restricciones.** Solo datos públicos de Generación Real del CEN en MWh, un período de ocho meses (sin estacionalidad anual) y sin información operacional (programación interna, mantenimientos).
- **Supuestos.** Los registros de 0 MWh son observaciones válidas mientras no exista evidencia de error; se mantiene la codificación horaria 1–24 del CEN con 24 observaciones por central y fecha; los datos publicados se consideran registros válidos de Generación Real del CEN.

3.5 Articulación con las fases F1–F4

Fase	Aporte principal	Evidencia
F1	Definición del problema, objetivos, alcance y entorno reproducible.	F1/F1_definicion.ipynb
F2	Carga, transformación, validación y dataset analítico en formato largo.	F2/F2_preparacion_datos.ipynb; src/
F3	Núcleo algorítmico, eficiencia, recursividad y arquitectura POO.	F3/F3_nucleo_algoritmico.ipynb; src/nucleo_poo.py
F4	Integración de resultados, visualizaciones, discusión, conclusiones y trazabilidad.	F4/F4_Consolidado_Proyecto.ipynb; changelog.md

IV. Aplicación de herramientas científicas

Se usa el ecosistema científico de Python: pandas para cargar, transformar y agregar los datos, NumPy para el cálculo numérico, Matplotlib para construir y exportar las figuras, y Jupyter para integrar código, resultados y explicación en un notebook por fase.

a. Reproducibilidad técnica

El proyecto se ejecuta en un entorno virtual dedicado con Python 3.14.7 y pandas 3.0.5 sobre Windows 11. Cada notebook (F1–F4) abre con una celda que imprime las versiones y verifica `sys.prefix != sys.base_prefix` para confirmar que el entorno virtual está activo. Las dependencias con versión fija se registran en `requirements.txt`, las rutas son relativas a la raíz del repositorio, los datos se separan en `data/raw` y `data/processed`, y `.gitignore` excluye entornos virtuales y archivos temporales, siguiendo las buenas prácticas de computación científica (Wilson et al., 2014). Los notebooks se entregan ejecutados desde kernel limpio (*Restart & Run All*), de modo que toda salida puede regenerarse.

b. Control de versiones

El historial se gestiona con Git y GitHub (Chacon & Straub, 2014) mediante ramas por tarea y fusión a `main` por *pull request*. Las identidades duplicadas de los cuatro integrantes se consolidaron con `.mailmap`, de modo que `git shortlog` muestra cuatro autores. Los cambios relevantes se resumen en `changelog.md` (fecha, descripción, hash, justificación y autor) y los hashes citados en este informe (Tabla 5) pueden verificarse con `git show <hash>`.

c. Documentación del proceso

`README.md` describe el problema, la fuente, la estructura del repositorio, la instalación, la ejecución, los resultados y las contribuciones por integrante y fase; `README_F3.md` detalla la arquitectura de F3. El código de `src/` documenta con docstrings sus clases y funciones públicas, y cada notebook explica en celdas Markdown las decisiones técnicas antes del código.

d. Repositorio GitHub actualizado

El repositorio público (URL en la portada) contiene un notebook por fase (F1–F4), el paquete `src/`, `data/`, `docs/` con este informe, `README.md`, `changelog.md` y `requirements.txt`. La rama `main` refleja la última versión integrada y no hay ramas remotas con trabajo pendiente de fusionar.

V. Diseño de soluciones algorítmicas eficientes

a. Codificación funcional

La lógica reutilizable vive en `src/` y los notebooks solo la invocan: `carga.py` lee el CSV del CEN; `transformacion.py` convierte los valores a numérico de forma estricta (`pd.to_numeric(errors="raise")`) y pasa de formato ancho a largo; `validacion.py` implementa reglas atómicas (sin nulos, sin duplicados, sin negativos, categorías esperadas y granularidad de 24 horas); `exportacion.py` guarda el dataset creando la carpeta de destino si no existe; y `secuencias.py` detecta secuencias de 0 MWh.

b. Preprocesamiento y transformación del dataset

El archivo crudo (359.891 filas × 33 columnas) se filtra al período 2026-01-01 a 2026-08-31 y a las tres centrales, quedando 729 registros anchos (243 días × 3 centrales). La transformación de las 24 columnas «Hora 1...24» a formato largo produce 17.496 observaciones × 13 variables (729 × 24). No se imputa ni elimina información: los 4.438 registros con 0 MWh (25,4 %) se conservan como valores válidos. La validación confirma 0 nulos, 0 IDs duplicados, 0 valores negativos y 729 combinaciones central-fecha completas.

c. Validación técnica y verificación del código

Las reglas de `validacion.py` se ejecutan sobre el dataset final en F2 y F3, y las pruebas con casos normales, límite y de excepción se concentran en F3 y F4. En F4, las agregaciones conservan el total de MWh (caso normal), funcionan con un único registro o solo con ceros (límite) y fallan con `KeyError` ante una columna ausente o `AssertionError` ante generación negativa (excepción). Además, el *Pipeline* orientado a objetos se reconstruye desde los datos crudos y se compara con el dataset de F2 mediante `pd.testing.assert_frame_equal`, confirmando equivalencia funcional.

d. Eficiencia y optimización

Las mediciones usan `timeit` y `tracemalloc` (Python Software Foundation, s. f.) y se resumen en la Tabla 1. Se mantiene la transformación vectorizada de pandas (The pandas development team, s. f.) por ser 5,3 veces más rápida y usar menos memoria que el bucle con `iterrows`. Para las secuencias de 0 MWh se adopta la versión iterativa: ambas son $O(n)$, pero la recursiva consume unas tres veces más memoria por la pila de llamadas y no es más rápida desde 250 observaciones; se conserva como contraste algorítmico.

Tabla 1. Comparación de eficiencia entre implementaciones equivalentes

Comparación	Implementación A	Implementación B	Resultado
Ancho → largo (17.496 obs.)	melt: 0,069 s; 4,40 MB	iterrows: 0,364 s; 11,13 MB	melt 5,3× más rápido; 2,5× menos memoria
Detección de ceros (17.496 obs.)	Iterativa: 0,0036 s; 154 KB	Vectorizada: 0,00034 s; 158 KB	Vectorizada ≈10× más rápida; memoria similar
Secuencias de 0 MWh (750 obs.)	Iterativa: 6.448 bytes	Recursiva: 18.952 bytes	Resultados idénticos; recursiva ≈3× más memoria

Fuente: elaboración propia, F3 (F3_nucleo_algoritmico.ipynb).

e. Diseño estructurado del código

El código se organiza por responsabilidad en el flujo carga → transformación → validación → agregación → exportación; los notebooks solo orquestan y narran. Cada función es breve, recibe sus parámetros (período, centrales, columnas horarias) de forma explícita y evita estado global, lo que facilita probarla de manera aislada.

f. Notebook final ejecutado y documentado

F4_Consolidado_Proyecto.ipynb integra el flujo completo: verifica el entorno, carga el dataset procesado, demuestra la equivalencia con el *Pipeline* de F3 reconstruido desde los datos crudos, ejecuta las pruebas de las agregaciones y genera las cinco figuras finales con un estilo común. Se entrega ejecutado desde kernel limpio, con las salidas visibles.

VI. Implementación de código modular y robusto

a. Programación orientada a objetos

src/nucleo_poo.py aplica tres principios sobre el esquema de clase base con ajustar/transformar de la guía del curso (Salinas Silva, 2026a). **Herencia:** FiltradorPeriodo, FiltradorCentrales, TransformadorAnchoLargo y ConstructorVariablesDerivadas heredan de Transformador, que fija el orden (ajustar antes de transformar) y deja a cada hija solo aprender() y aplicar(). **Polimorfismo:** Pipeline recorre sus pasos con la misma llamada, paso.ajustar(df).transformar(df), sin preguntar de qué clase es cada uno. **Encapsulamiento:** cada objeto guarda su estado en atributos internos (_parametros, _ajustado) y transformar lanza RuntimeError si no se ajustó antes.

src/agregacion_temporal.py aplica el patrón *Strategy* (Gamma et al., 1994). Sin él, calcular perfiles por hora, día, mes y día de la semana habría requerido funciones casi idénticas, y cada criterio nuevo obligaba a modificar código ya probado. El patrón encapsula cada criterio en una clase con la misma interfaz agregar() (AgregacionHoraria, AgregacionDiaria, AgregacionMensual y AgregacionPorDiaSemana; 72, 729, 24 y 21 resultados) y CaracterizadorTemporal las usa de forma intercambiable sin conocerlas.

b. Documentación de arquitectura

La Tabla 2 resume la arquitectura. Los notebooks importan desde src/ y ningún módulo depende de un notebook; los datos fluyen de data/raw al *Pipeline*, luego a la validación, a data/processed y finalmente a las agregaciones y figuras.

Tabla 2. Arquitectura del paquete src/

Módulo	Responsabilidad	Componentes principales
carga.py	Lectura del CSV del CEN	cargar_datos_cen
transformacion.py	Conversión numérica y ancho → largo	Funciones de transformación

Módulo	Responsabilidad	Componentes principales
<code>validacion.py</code>	Reglas de calidad atómicas y parametrizables	<code>validar_sin_nulos</code> , <code>validar_no_negativos</code> , ...
<code>nucleo_poo.py</code>	Preparación orientada a objetos	Transformador y subclases, Pipeline
<code>agregacion_temporal.py</code>	Agregaciones intercambiables (<i>Strategy</i>)	Agregacion*, CaracterizadorTemporal
<code>secuencias.py</code>	Secuencias de 0 MWh (iterativa y recursiva)	<code>secuencias_cero *</code> , <code>comparar_implementaciones</code>
<code>exportacion.py</code>	Escritura del dataset procesado	<code>exportar_dataset</code>

Fuente: elaboración propia a partir del repositorio del proyecto.

VII. Construcción de visualizaciones de datos efectivas

a. Preprocesamiento y transformación de datos

Las figuras se construyen sobre el dataset procesado, sin transformaciones adicionales que alteren los datos. Las agregaciones (media horaria, suma diaria, media mensual, media por día de la semana y porcentaje de horas con 0 MWh) se calculan con pandas (McKinney, 2010) y las estrategias de `agregacion_temporal.py` y se verificó que conservan el total de MWh. Se usan medias para comparar centrales de escala distinta, los ejes parten en cero y los registros de 0 MWh se muestran, no se excluyen. Sin codificación ni escalamiento, el dataset conserva sus categorías y la unidad original (MWh), y se grafica directamente.

b. Visualizaciones analíticas

De las cinco figuras del notebook F4, aquí se muestran las tres que sustentan los resultados. Cada título enuncia el hallazgo, los colores son consistentes por central y cada figura declara su fuente.

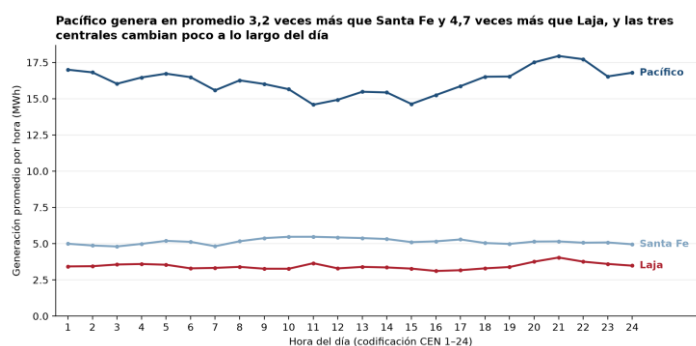


Figura 1. Perfil horario medio de generación por central. Fuente: elaboración propia con datos del CEN (2026).

La figura muestra que las tres centrales mantienen perfiles horarios relativamente planos, con Pacífico en un nivel medio claramente superior a Laja y Santa Fe. Se infiere que, dentro del período analizado, la hora del día explica menos variación que las diferencias entre centrales. Esta lectura se limita a la Generación Real del período y no permite atribuir causas operacionales. Como contexto del relato, este resultado orienta el análisis hacia una escala temporal mayor: la comparación mensual de la Figura 2.

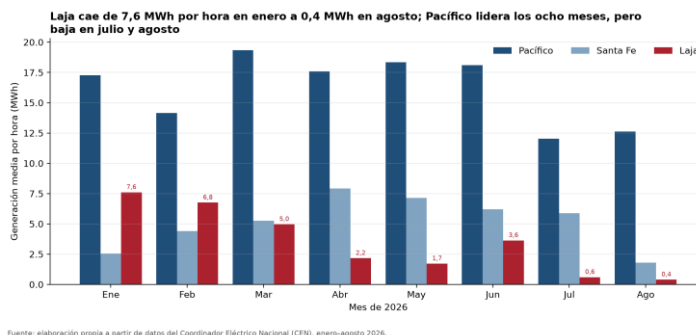


Figura 2. Generación media mensual por central. Fuente: elaboración propia con datos del CEN (2026).

La figura muestra que las diferencias entre centrales se amplían a escala mensual: Laja disminuye hacia agosto, Pacífico reduce su nivel en julio-agosto y Santa Fe alcanza sus mayores medias en el período intermedio. Se infiere que la variación mensual discrimina mejor los cambios temporales que el perfil horario casi plano observado previamente. La comparación cubre solo ocho meses, por lo que no permite establecer estacionalidad anual ni explicar las causas de las variaciones. Este contraste conduce a examinar en la Figura 3 cuánto de esas diferencias coincide con la presencia de horas registradas en 0 MWh.

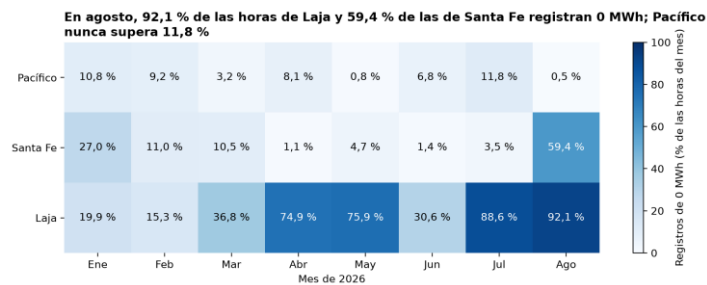


Figura 3. Porcentaje de horas con 0 MWh por central y mes. Fuente: elaboración propia con datos del CEN (2026).

La figura muestra que los registros de 0 MWh se concentran principalmente en Laja, especialmente en julio y agosto, mientras Pacífico presenta porcentajes bajos y Santa Fe aumenta en agosto. Se infiere que la frecuencia de ceros contribuye a describir las diferencias temporales observadas entre las centrales y ayuda a interpretar la caída mensual de Laja. Sin información operacional complementaria, estos ceros no pueden atribuirse a fallas, mantenimiento, detenciones u otra causa específica. La figura cierra el relato y responde a los objetivos de caracterizar, comparar y describir los patrones temporales.

VIII. Metodología del desarrollo técnico

Las fases se encadenan: F1 definió la pregunta y el entorno, F2 produjo el dataset validado, F3 llevó la preparación a clases y midió la eficiencia, y F4 interpretó y comunicó los resultados. El flujo es `Pipeline.ajustar().transformar()` → `validar_dataset_procesado()` → `CaracterizadorTemporal.caracterizar()`. Cada decisión produjo un resultado: melt, 5,3 veces más rápido que iterrows, reprocesa todo el período; las reglas atómicas se prueban por separado; y conservar los 0 MWh hizo posible la Figura 3.

a. Validación y reflexión técnica

La validación técnica (sección V.c) confirmó la equivalencia entre F2 y F3 y la conservación de totales en las agregaciones. Entre las limitaciones, el período cubre ocho meses (no permite evaluar estacionalidad anual), los valores son registros de Generación Real del CEN, es decir, generación inyectada y no producción total de cada planta, y los datos no incluyen información operacional que explique los registros de 0 MWh. Como aprendizaje, llevar la lógica a `src/` con reglas atómicas redujo la duplicación y facilitó detectar errores.

La reflexión metodológica de la Fase 4 revisó críticamente decisiones técnicas previas, como la refactorización de `validacion.py` en reglas atómicas y parametrizables (commit a4e655d), que fortalece la modularidad y permite que la prueba de excepción de F4 use solo `validar_no_negativos()`. La reflexión no originó esta mejora: sirvió para evaluar su aporte a la reproducibilidad, la verificabilidad y la trazabilidad. En escalabilidad, sumar meses o centrales solo cambia los parámetros de `FiltradorPeriodo` y `FiltradorCentrales`. La colaboración se organizó con ramas por tarea y pull requests a main.

Tabla 3. Comparación con las fases previas

Aspecto	Fase previa	Estado actual	Motivo del cambio
Preparación	Funciones sueltas en el notebook de F2	Clases y Pipeline en <code>src/</code> , con equivalencia verificada	Modularidad y reutilización
Validación	Una función con valores rígidos	Reglas atómicas y parametrizables	Probar cada regla por separado
Eficiencia	Sin mediciones	<code>timeit</code> y <code>tracemalloc</code> en F3	Justificar la implementación elegida

Aspecto	Fase previa	Estado actual	Motivo del cambio
Verificación	Equivalencia columna a columna (F3)	<code>assert_frame_equal</code> y pruebas normal/límite/excepción (F4)	Evidencia objetiva de que las mejoras no rompen nada

Fuente: elaboración propia a partir de los notebooks F2–F4 y de las observaciones recibidas.

b. Resultados

La Tabla 4 resume los resultados. TER CMPC Pacífico aporta el 65 % de la generación total del conjunto y tiene el menor porcentaje de horas en cero (6,4 %). TER CMPC Laja disminuye de 7,6 MWh por hora en enero a 0,4 MWh en agosto (Figura 2) y concentra el 72 % de las horas con 0 MWh del conjunto, con una secuencia continua máxima de 375 horas ($\approx 15,6$ días). TER CMPC Santa Fe alcanza su mayor media en abril (7,9 MWh) y llega a 59 % de horas en cero en agosto (Figura 3).

Tabla 4. Resumen de generación y registros con 0 MWh por central (enero–agosto de 2026)

Central	Media horaria (MWh)	Máximo horario (MWh)	Total (MWh)	Horas con 0 MWh	Secuencia máx. de 0 MWh
TER CMPC Laja	3,45	27,3	20.096	3.193 (54,7 %)	375 h
TER CMPC Pacífico	16,19	33,2	94.446	371 (6,4 %)	57 h
TER CMPC Santa Fe	5,14	17,9	29.976	874 (15,0 %)	356 h

Fuente: elaboración propia con datos del CEN (2026); secuencias calculadas con `secuencias.py`.

c. Discusión

Los resultados muestran tres comportamientos distintos: Pacífico presenta una generación comparativamente alta y estable; Laja, una disminución marcada y una alta frecuencia de registros de 0 MWh; y Santa Fe, un nivel intermedio con períodos prolongados de registros consecutivos de 0 MWh. La ausencia de un ciclo horario marcado es compatible con una generación de excedentes sujeta a la programación interna de los procesos, pero los datos disponibles no permiten confirmarlo ni atribuir los 0 MWh a detenciones, mantenimiento u otras causas, por lo que se mantiene como hipótesis fuera del alcance. La principal amenaza a la validez es el período de ocho meses, que no distingue tendencia de estacionalidad.

d. Conclusiones

Respecto de la pregunta de investigación, los patrones temporales se resumen en cuatro conclusiones:

- La generación horaria es casi plana en las tres centrales, sin ciclo diario marcado, y Pacífico opera a un nivel 3 a 5 veces superior al de Laja y Santa Fe.
- A escala mensual, Laja decrece de forma sostenida y Pacífico baja en julio y agosto, mientras Santa Fe alcanza su máximo en abril–mayo.
- Los registros de 0 MWh (25,4 % del total) se concentran en Laja y se agrupan en secuencias largas, es decir, en tramos continuos y no en registros aislados.
- El flujo es reproducible y verificable: pipeline modular con pruebas, equivalencia $F2 \leftrightarrow F3$ y trazabilidad en Git. Como trabajo futuro, ampliar el período y contrastar los ceros con información operacional.

e. Trazabilidad de mejoras

Cada observación se tradujo en una acción con su commit (Tabla 5), con más modularidad (siete módulos en `src/`), rendimiento medido y documentación verificable en `changelog.md`.

Tabla 5. Trazabilidad de observaciones y acciones de mejora

Observación recibida	Acción realizada	Evidencia / commit	Impacto técnico
Modularización insuficiente del pipeline	Se trasladó lógica reutilizable desde los notebooks hacia módulos especializados y se amplió la arquitectura de src/	550cfc6, 2a7372f	Menor duplicación, mejor separación de responsabilidades y mayor mantenibilidad
Validación concentrada y con valores rígidos	Se refactorizó validacion.py en reglas atómicas y parametrizadas	a4e655d	Cada regla puede probarse de forma independiente y reutilizarse con distintos períodos o conjuntos de datos
Identidades Git inconsistentes	Se incorporó y corrigió .mailmap para consolidar las identidades del equipo	5e60e7f, ce134a7	Mejora la atribución y trazabilidad de las contribuciones
Notebooks sin evidencia completa de ejecución desde kernel limpio	Se corrigió y verificó la ejecución reproducible de F1 y F2; posteriormente F3 también fue validado desde kernel reiniciado	fd08c0f	Mayor consistencia entre lo declarado en el informe y los artefactos versionados
Falta de análisis de eficiencia	Se incorporaron benchmarks y comparación entre implementaciones	1d67d9b	Las decisiones algorítmicas quedan sustentadas con evidencia empírica
Necesidad de integrar la lógica previa bajo POO	Se refactorizó el pipeline e incorporó filtrado temporal dentro de la arquitectura orientada a objetos	2a7372f	Mayor cohesión, extensibilidad y reutilización del flujo
Falta de trazabilidad consolidada entre fases	Se creó changelog.md con fecha, descripción, commit, justificación e impacto	8f9f73e	Permite reconstruir la evolución técnica del proyecto entre F1 y F4
Visualizaciones finales sin fuente e interpretación explícita	Se documentaron individualmente las tres figuras finales con fuente e interpretación analítica	9158bda, c7aaee2, 9cdf8c7	Mejora la comunicación de resultados y la relación entre cada figura y los objetivos del proyecto

IX. Presentación Audiovisual

El video de 5 a 8 minutos presenta el contexto, la metodología, los resultados y las conclusiones del proyecto, con participación de los cuatro integrantes. Enlace al video: <https://youtu.be/SCsrixbw2E>

X. Bibliografía

- Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. <https://git-scm.com/book>
- Coordinador Eléctrico Nacional. (2022). *Diccionario de datos para la descarga de datos Portal Web Sistema de Información Pública (SIP)*, B43-DIN-04.
- Coordinador Eléctrico Nacional. (2026). *Generación real* [Conjunto de datos]. Portal de Operación Real del Sistema Eléctrico Nacional. <https://www.coordinador.cl/operacion/graficos/operacion-real/generacion-real/>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.
- McKinney, W. (2010). Data structures for statistical computing in Python. En S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- Python Software Foundation. (s. f.). *timeit y tracemalloc*. En *The Python standard library*. <https://docs.python.org/3/library/timeit.html>
- Salinas Silva, O. (2026a). *Guía de apoyo Sumativa 3 – Fase 4* [Documento del curso MCDI500]. Magíster en Ciencia de Datos e Inteligencia Artificial, Universidad Andrés Bello.
- Salinas Silva, O. (2026b). *Programación para la ciencia de datos* [Presentación de clase, Semana 1]. Magíster en Ciencia de Datos e Inteligencia Artificial, Universidad Andrés Bello.
- The pandas development team. (s. f.). *pandas documentation*. <https://pandas.pydata.org/docs/>
- Wilson, G., Aruliah, D. A., Brown, C. T., Chue Hong, N. P., Davis, M., Guy, R. T., Haddock, S. H. D., Huff, K. D., Mitchell, I. M., Plumbley, M. D., Waugh, B., White, E. P., & Wilson, P. (2014). Best practices for scientific computing. *PLoS Biology*, 12(1), e1001745. <https://doi.org/10.1371/journal.pbio.1001745>